

Libri mette la roba nel Tea

① Algorithms on strings, trees and sequences

② Randomized Algorithms

③ Compact Data Structure

↳ poco spazio
ENORMI quantità di dati $\rightarrow$ si opera su dati compressi

Modalità di Esame

Classica

Approfondimento $\rightarrow$ proponi o diedi di proporre

(qua si indicizza a 1 non a 0: tranne in alcuni casi)

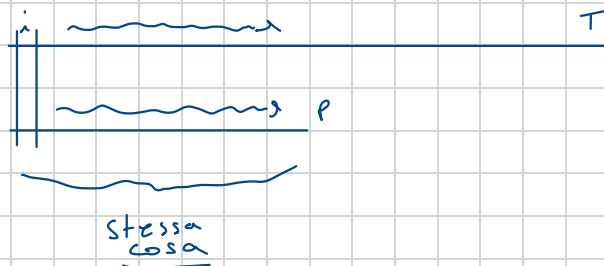
Problema: PATTERN MATCHING ESATTO

Input: $P, T \in \Sigma^*$ ^{stringhe}
 $P = \text{patte}$ $T = \text{testo}$
stringhe nell'alfabeto Σ (successioni finite di caratteri)

$|P| = n$
 $|T| = m$

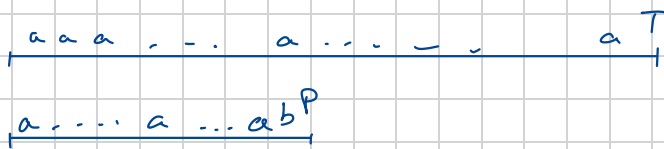
collezione finita di caratteri (nel nostro caso $|\Sigma| \geq 2$)

OUTPUT $\{i \mid T[i, i+n-1] = P\}$
 $T[i, j]$ slice di T da i a j
 $T[i, \infty]$ da i in poi



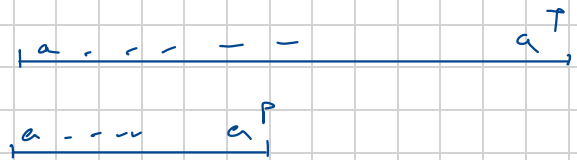
Un algoritmo naïve è $O(m \cdot n)$ $\rightarrow$ confronto semplice

quale è il caso peggiore?



devo scansionare tutto

↓
0 occorrenze



tutte le occorrenze

Algoritmo Naïve

for ($i \leftarrow 1$ to $m - n + 1$) do

for ($j \leftarrow 1$ to n) do

if $T[i, i+j-1] = P[j, j]$ then exit

end for

if $T[i, i+n-1] = P$ then output

end for

$O(m \cdot n)$

Dove è debole l'algoritmo naïve?

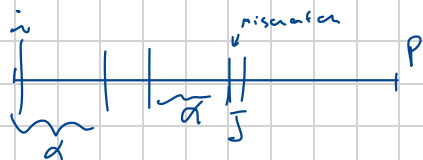
Proviamo a vedere in questa situazione:



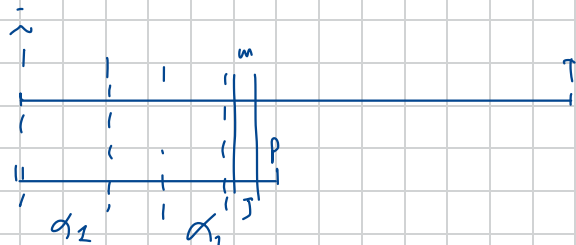
riesco a sfruttare la parte prima del mismatch?

↙
↘

Algoritmo Knuth-Morris-Pratt



α max prefisso suffisso di $P[i..J]$



posso ripartire da



riparto da qua

questa è la prima occorrenza, perché se no avrei trovato un α più grande.

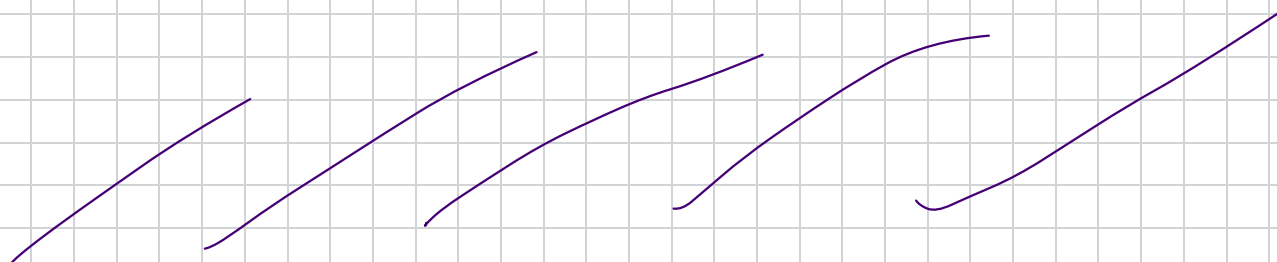
continuando...

per ogni J ho bisogno del max pre/suf di $P[i..J]$

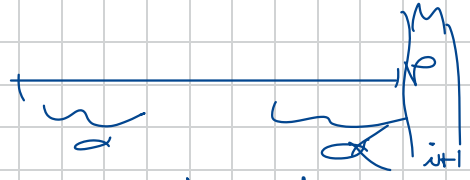
Osservazione: utilizzando $\uparrow$ modifico l'algoritmo naïve facendo in modo che ad ogni passo avanti i o avanti J

$\searrow$ $O(m+n)$ + costo per $\text{crack}(\text{max pre/suf})$

$\searrow$ Levenshtein



$sp_i(P)$ max prefisso^{suffisso} di $P[1, i]$

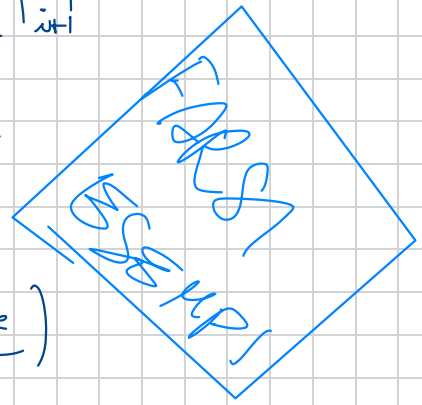


se sono in grado di computare $sp_i(P)$ per ogni $i \in \{1, \dots, |P|\}$ in tempo lineare, allora posso modificare l'algo Naive $\rightarrow$ KMP

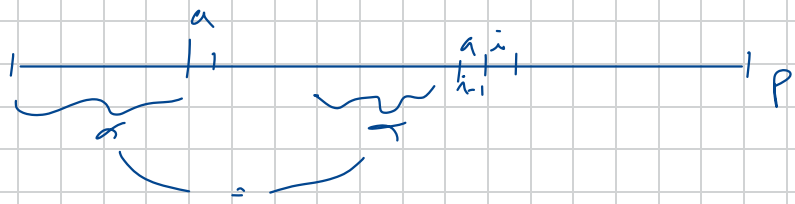
devo fare un pre processing del pattern

$O(n + m)$

lower bound (banale)



1 metodo $sp_i(P)$



se $P[sp_{i-1}(P) + 1] \neq P[i]$ allora:

$sp_i(P) = sp_{i-1}(P) + 1$

come lineare?

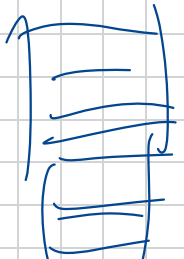
COMPLESSITA' AMMORTIZZATA

Se $P[sp_{i-1}(P) + 1] \neq P[i]$ devo eseguire una chiamata ricorsiva in quanto il max prefisso/suffisso di $P[1, i]$ sarà il max pre/suff di $P[1, sp_{i-1}(P)]$. Questo significa che nel calcolo di $sp_i(P)$ posso effettuare fino ad "i" chiamate ricorsive

analisi ammortizzata

quando procedo verso destra nel caso $P[sp_{i-1}(P) + 1] = P[i]$ all'occorrenza il doppio del costo

esempio pi la (x compl. ammortizzata)



push $O(1)$

multi-pop $O(n)$

ris. quadratic

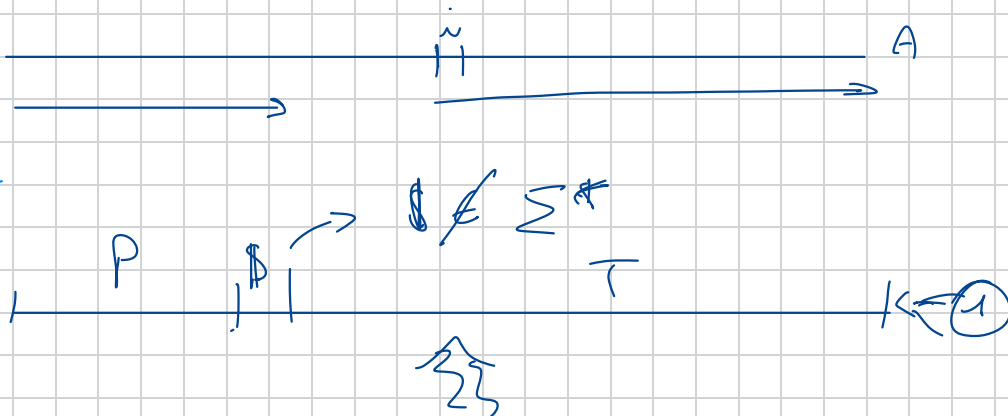
tecnicamente multi-pop può avere complessità fino ad n^2 , ma se io pago 2 per ogni push pago già per il pop $\rightarrow$ lineare

Algoritmo Z

Data una stringa A definiamo

$$Z_i(A) = \begin{cases} 0 & \text{se } i=0 \vee A[1] \neq A[i] \\ \max \{ j > 0 \mid A[1, j] = A[i, i+j-1] \} \end{cases}$$

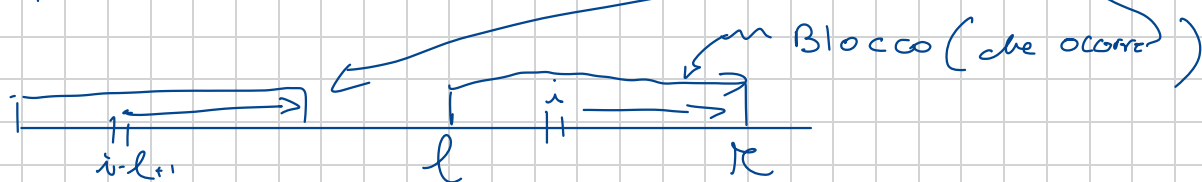
- Come la calcolo?
- Quanto mi costa?



$$Z_i(\underbrace{P \parallel T}_A) = u \quad \text{se } i \text{ è una posizione in cui } P \text{ occorre in } T$$

Quindi come calcolo $Z_i(A)$?

$$Z_1(A) = \text{tutto } A$$



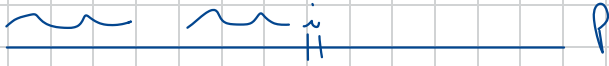
Il calcolo di Z , a parte il caso banale iniziale per $Z_1(A)$, procede mantenendo il Blocco $B[l, r]$ che corrisponde $r = l + Z_l(A) - 1$ max (il blocco con estremo dx massimo)

Dovrò aggiornare l ed r quando necessario

Facendo riferimento a ①, per calcolare $Z_i(A)$ andrò in posizione $i-l+1$ e gestirò i casi:

- se $Z_{i-l+1}(A) < r-l$ allora $Z_i(A) = Z_{i-l+1}(A)$
 altrimenti riprendo con i confronti tra $A[r+j]$ e $A[i-l+j+1]$ finché trovo dei match
 in uscita aggiorno $l=i$ ed $r=i+Z_i(A)$

Riduzione di $sp_i(P)$ ad una Z computazione



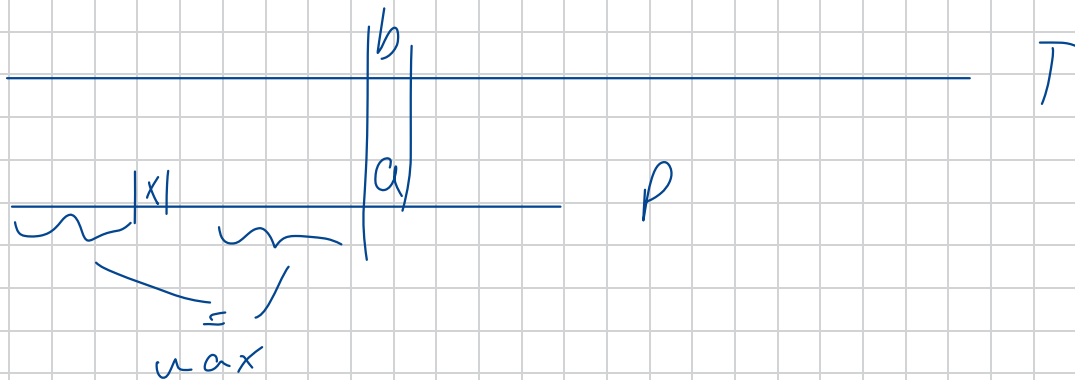
Def: J mappa su i sse $i = J + Z_J(P) - 1$

Osservazione $\forall i \in \{1, \dots, |P|\}$ $sp_i(P) = Z_J$ sse J è il max che mappa su i

Esercizio: Computare $sp_i(P)$ per ogni $i \in \{1, \dots, |P|\}$ usando Z calcolate su P

KMP è superiore a Z

$\neg \left(\left(\text{può essere online (cerca senza avere tutto l'input)} \right) \wedge \underline{\text{No}} \right)$

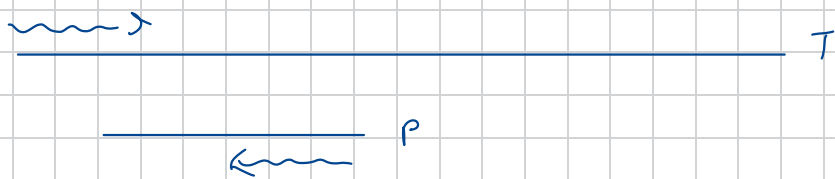


• $sp'_i(P)$ è il max pref/suff di $P[1, i]$ t.c. $P[sp'_i(P)+1] \neq P[i]$

• $sp_{i,x}(P)$ è il max pre/suff di $P[1, i]$ b.c. $P[sp_{i,x}(P)+1] = x$ con $x \in \Sigma$

KMP è superiore a Z in senso real-time

Algoritmo di Boyer - Moore ultimo alga "stupido"



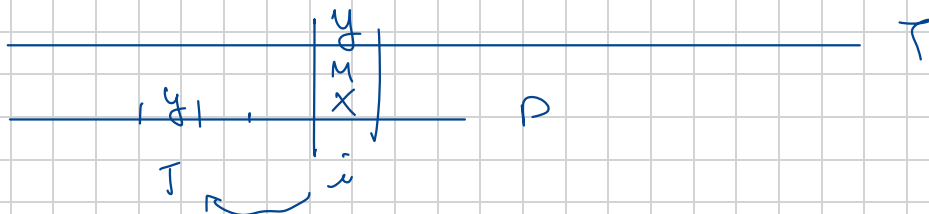
scandiamo T da sx vs dx
P da dx vs sx

m passi per scandire $T = 2 \frac{m}{u}$

sublineare

se in P occorrono caratteri a bassa frequenza in T

(BCR) Bad Character Rule: per ogni carattere di $y \in \Sigma$ e ogni posizione $i \in \{1, \dots, |T|\}$ voglio la prima posizione $j < i$ t.c. $P[j] = y$



Good Suffix Rule: per ogni $i \in \{1, \dots, |P|\}$ max j tale che

$$P[j-n+1, j] = P[i, |P|]$$

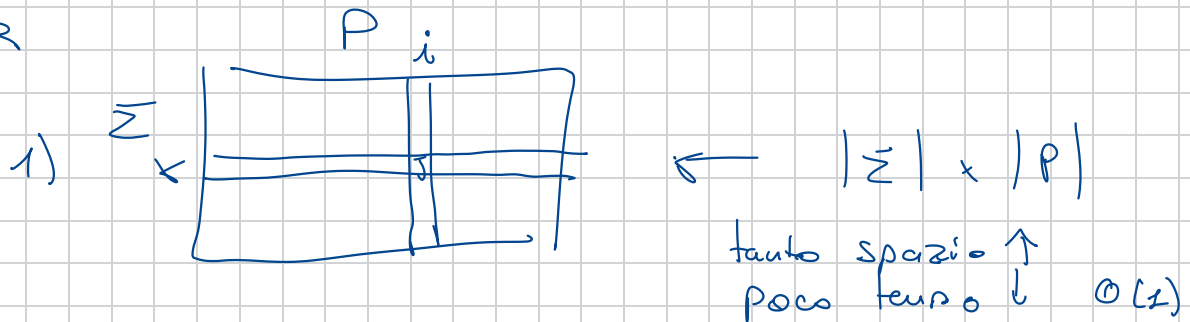


Per implementare sta roba uso Z

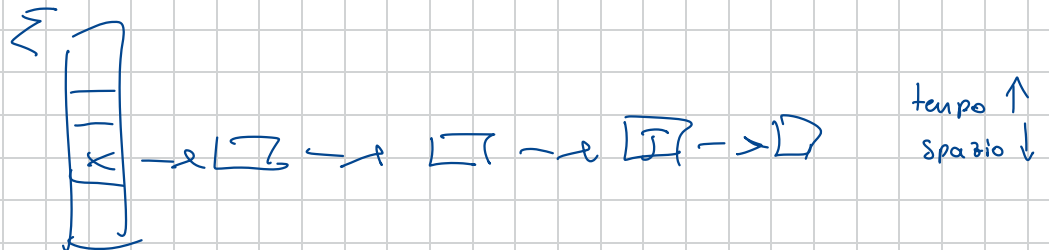
Una volta implementate BCR e GSR risolve il problema del pattern matching esatto con un loop che scandisce T da sr a dx , P da sr a sr e dopo un mismatch o un fine P applica la regola che ci consente di riposizionare P il più a dx possibile

Idee per l'implementazione delle 2 rule

• BCR



2) linked list



analisi complessità annunziata come?
 $\hookrightarrow$ tempo $O(1)$

• GSR si implementa con Z (vedi anche Gusfield)

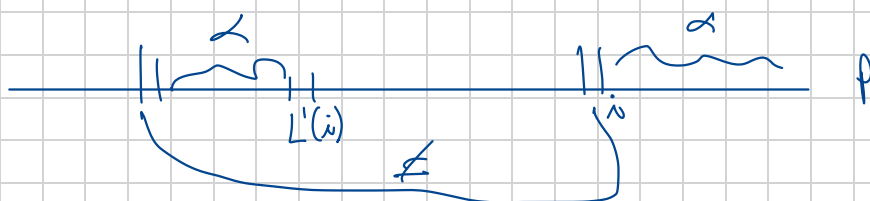
$\hookrightarrow$ su P^r (P reverse)

Def: $P \in \Sigma^*$, $i \in \{1, \dots, |P|\}$

$L'(i)$ è il massimo indice f.c.

1) il suffisso $P[1, n]$ è anche un suffisso di $P[1, L'(i)]$

2) $P[i-1] \neq P[L'(i) - (n-i) - 1]$



Def: $N_j[P]$ la lunghezza del max suffisso di $P[1, j]$ che è anche suffisso di P

[Teo: $L'(i)$ è il massimo j t.c. $N_j(P) = |P[i, n]| = n - i + 1$

$$N_j(P) = Z_j[P^R]$$

$\Theta(n \cdot m)$
Sufficiente
in caso
OK

Problema diverso da Pattern Matching Esatto ma particolarmente utile viene dai DataBase.

T ← base di dati / testo $\mathcal{P} = \{P_1, \dots, P_z\}$ ← querele collezione di pattern ← input

output: tutte le occorrenze di ogni $P_i \in \mathcal{P}$ in T

Soluzione Naive uso KMP o Z o Boyer-Moore su ognuno dei pattern in $\mathcal{P}$

costo per un pattern $\Theta(|T| + |P_i|)$ con $P_i \in \mathcal{P}$

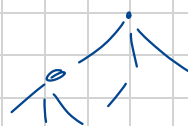
costo totale $\Theta(z \cdot m + n)$

$$|T| \quad \searrow \quad \sum_{i=1}^z |P_i|$$

Dobbiamo fare meglio, come? non leggendo il test ogni volta
↓ ??

devo immaginare informazione → Algo Aho-Corrick

↳ serve un albero



↓
[SCRIVO
MEGLIO]

Serve un albero che ci consenta di abbassare la complessità del fattore α

Def : (Keyword Tree)

Def Dato $P = \{P_1, \dots, P_k\}$ dato un Keyword Tree di P è un albero $K(P)$ etichettato con elementi di Σ sugli archi, t.c.

- 1.) Archi uscenti dallo stesso nodo hanno etichette diverse
- 2.) Si può leggere ogni pattern $P_i \in P$ partendo dalla radice e andando verso la foglia
- 3.) Ogni foglia corrisponde ad un $P_i \in P$

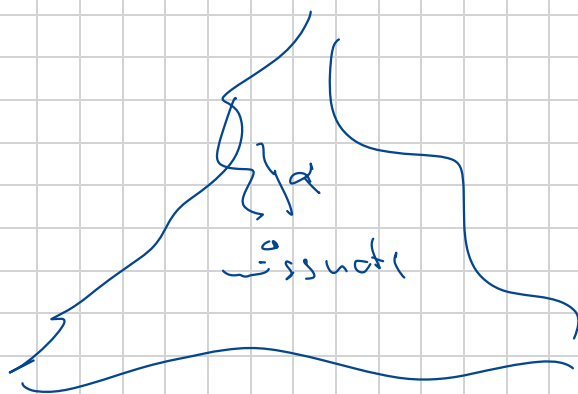
Iniziamo trattando il caso (più semplice) in cui nessun $P_i \in P$ occorre in $P_j \in P$ per $P_i \neq P_j$ ma nessun P_i è prefisso

Dimensione di $K(P)$ numero di archi = n° nodi - 1 = $\sum_{i=1}^k |P_i| = n$

Altezza di $K(P)$ lunghezza del pattern più lungo $\max\{|P_i| : P_i \in P\}$

$\text{fan-out} \leq |\Sigma| \leftarrow$ n° max di figli per un nodo.

Problema



Pensaci

poi α deve ripartire dalla radice

"Un lower bound è il tempo necessario per scrivere l'output"

13/03/2026

Algoritmo di Aho-Corasick

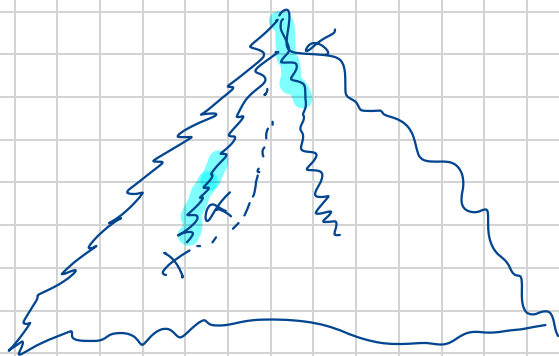
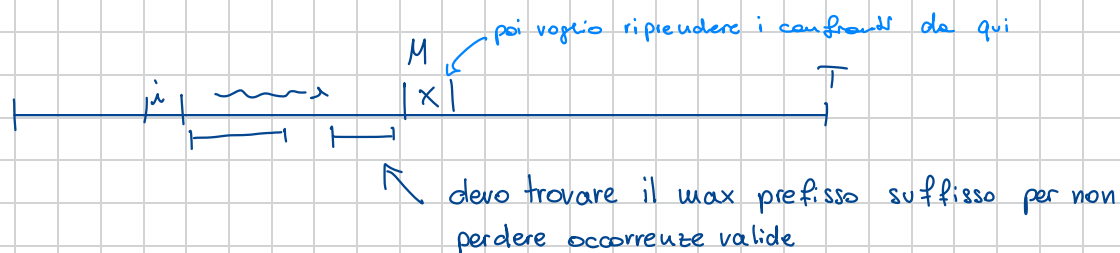
T $P = \{p_1, \dots, p_z\}$

Come trovo tutte le occorrenze di $p_i \in P$ in T per $i \in \{1, \dots, z\}$

Obiettivo: evitare di scandire il testo ogni volta $\Rightarrow O(|T| + \sum_{i=1}^z |p_i|)$

$\Rightarrow$ Ci vuole un albero che ci consenta di "tenere in memoria" del lavoro fatto

Come si costruisce K ? Cosa mi serve oltre la descrizione carattere per carattere dei $p_i \in P$.



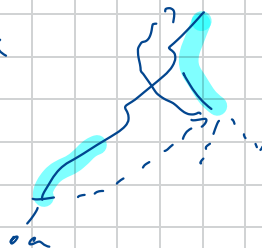
Voglio il max α che sia:

- 1) Suffisso della stringa che ho letto
- 2) prefisso di una qualche stringa in P .

devo aggiungere dei puntatori (così diventa simil automi)

FAILURE LINK

come calcolerò i failure link in fase di preprocessing?



(complessità ammortizzata)

Formalizziamo la nozione di failure link

Def. Dato $v \in K(P)$ dico:

- $L(v)$ stringa delle etichette del cammino dalla radice a v
- u_v nodo di $K(P)$ t.c. $L(u_v)$ è il max prefisso suffisso di $L(v)$
- $lp(v) = |L(u_v)|$ profondità di u_v

esiste 1 unico

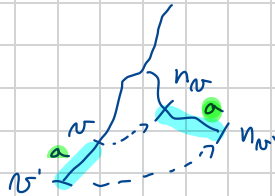
$$\forall n \in K(\mathcal{P}) \exists! n_n$$

Osservazione: n_{15} potrebbe essere la radice di $K(P)$

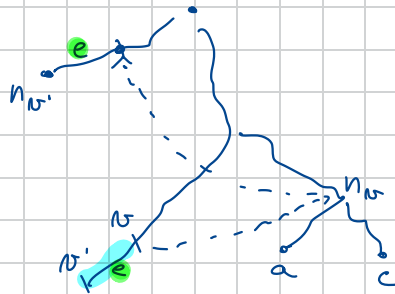
Def: $\langle n, n_n \rangle$ est un failure link

L'algoritmo per completare $K(P)$ con i failure link sarà un algoritmo ricorsivo che processa i nodi per profondità crescente; due casi:

i) c'è un arco etichettato con il carattere dell'arco che raggiunge n' uscente da n .

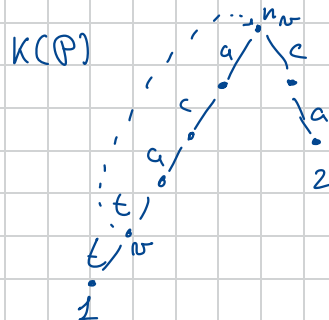
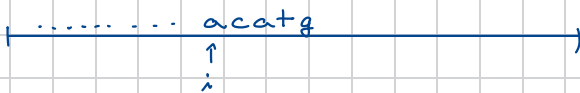


2) !1) Possono essere necessari un numero di passi pari (asintoticamente) all'altezza di $k(P)$



Complessità: Lineare mediante analisi ammortizzata.

Esercizio: $P = \{ \text{acatt}, \text{ca} \}$

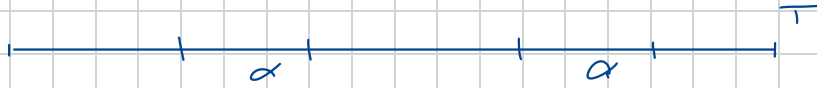


In questo esempio dopo la scansione a partire da i scendo fino alla prima t ed il failure link mi rimanda alla radice. Perdo l'occorrenza di P_2 in posizione $i+1$

Soluzione: output link de finisci te

(substring $\neq$ subsequence)

Longest Repeated Substring



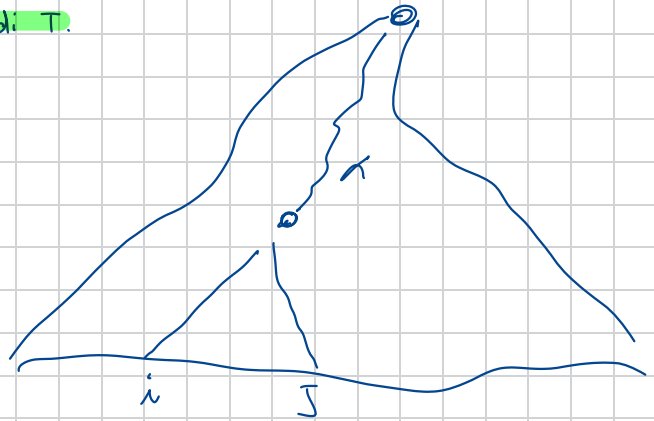
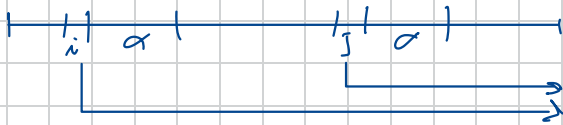
Dato un testo T , produrre una sottostringa di T ripetuta (≥ 2 occorrenze) di lunghezza massima

Non è facile da risolvere in tempo subquadratico

Idea: Usare una struttura dati che generalizza i Keyword Tree

Dato un testo $T \rightsquigarrow T^i = T[1, i]$ i -esimo prefisso
 $\rightsquigarrow T_i = T[i, |T|]$ i -esimo suffisso.

Suffix Trie è il Keyword Tree dei suffissi di T .



Side Quest: si può usare per risolvere il problema precedente? sì

16/03/2026

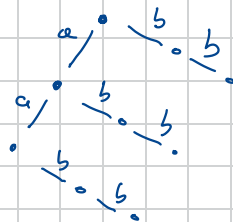
Il caso pessimo per un suffix trie (keyword tree della collezione dei suffissi di T) non può essere peggio che quadratico ($O(|T|^2)$)

Come costruisco un keyword tree?

↓

Parto dal più lungo e poi man mano costruisco l'intero albero.

esempio: di suffix trie $\Sigma = \{a, b\}$ $aa \dots aa bb \dots bb$



VIENE QUADRATICO

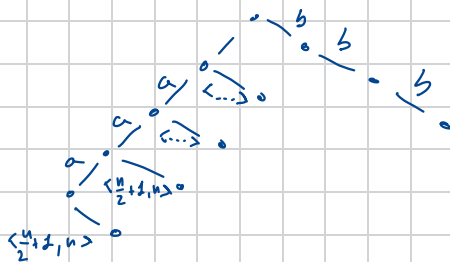
Suffix Trie $\xrightarrow[\text{cammini "obbligati"}]{\text{elimino i}} \text{Suffix Tree}$

E' possibile produrre T t.c. il suffix trie di T ha dimensione $\in \Theta(|T|^2) : a^{n/2} b^{n/2}$

↳ se voglio una struttura dati che memorizzi tutti i suffissi di T in spazio lineare devo modificare la definizione di Trie

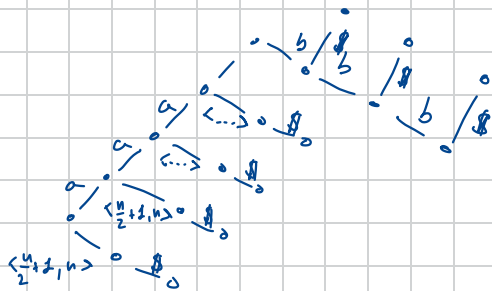
Idea: Elimina tutti i cammini massimali fatti di nodi con un solo figlio ed etichetta l'arco che sostituisce il cammino, con una coppia di indici che identificano la stringa che etichetta il cammino nel testo.

esempio: $a^{n/2} b^{n/2}$



↳ devo mantenere i nodi che corrispondono ai suffissi.

$\$ \Sigma$ mi consente di escludere il caso in cui un suffisso e' sottostringa prefisso o fattore di un altro.



N.B. Devo dimostrare che i Suffix Tree hanno dimensione lineare in $|T|$

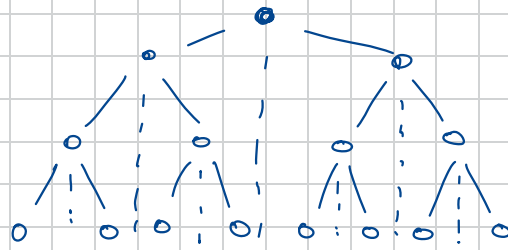
Lemma:

Sia T un albero privo di nodi u, v, w t.c. $u - v - w$ e' un cammino in T e v e' il solo figlio di u e w e' il solo figlio di v (Sia T t.c. ogni nodo interno e' branching (ha almeno 2 figli)) allora $|T| \in O(|T|)$

Dim:

↳ ogni nodo può avere 0, 1 o 2 figli

Assumo T binario e dimostro che il numero di nodi interni (nodi con 1 o 2 figli) e' $\leq$ del numero di foglie (nodi con 0 figli) - 1



Per induzione sul numero di foglie n : nodi i : no nodi interni f : foglie

$f = 1$ $i = 0$ $\rightarrow$ c'è solo la radice

$f > 1$ x nodo interno di profondità massima, x ha due figli e devono essere entrambi foglie. Cancello le due foglie figlie di x e ottengo un albero T' con una foglia in meno (perché x diventa foglia) e un nodo interno in meno

$i' = f' - 1 \rightarrow i = i' + 1$ $f = f' + 1 \rightarrow i = f - 1$

□

Algoritmo di Ukkonen

- 1) dimostra che è possibile costruire il Suffix Trie di T in tempo ottimale. ^{tempo necessario a scrivere l'output} Riscrive il problema come la costruzione di un automa a stati finiti esteso
- 2) adatta la precedente costruzione al caso dei Suffix Tree

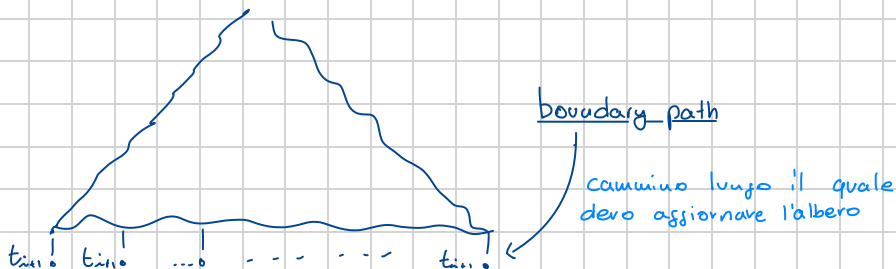
Costruzione di un Suffix Trie



Idea di Ukkonen per 1)

i) immagino di avere Suffix Trie (T^i) ^{prefisso i-esimo di T} che sarà un automa esteso

ii) descrivo una strategia per appendere $t_{i+1} = T[i+1]$ ad ognuno dei suffissi in Suffix_Trie (T^i) $\Rightarrow$ Suffix_Trie (T^{i+1})

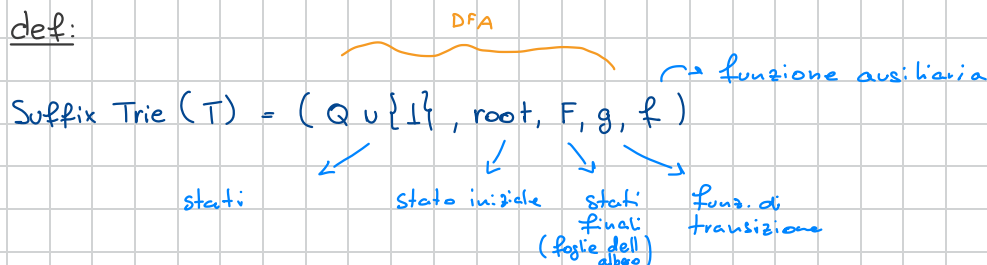


NOTAZIONE

$T = t_1 t_2 \dots t_n$ $T^i = t_1 \dots t_i$ $T_i = t_i \dots t_n$ $\left\{ \begin{array}{l} \text{prefisso} \\ \text{suffisso} \end{array} \right\}$ i -esimo

$S(T) \rightarrow$ insieme dei suffissi di T

def:



- Se x è una stringa di caratteri $\rightarrow \bar{x}$ è lo stato (associato ad x)
- $g(\bar{x}, a) = \bar{y} = \overline{xa}$ $\leftarrow$ mi dice come muovermi
- $f(\bar{x}y) = \bar{y}$ $\leftarrow$ verrà usata per muoversi da un suffisso all'altro durante la fase di aggiornamento
- $f(\text{root}) = \perp$

suffix link (corrisponde al failure link dei Keyword Tree)

$$\delta(T^{i+1}) = \delta(T^i) t_{i+1} \cup \{E\}$$

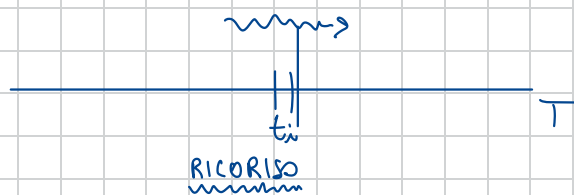
DAI UN OCCHIO AL LAVORO COMMENTATO DEL PROF. PER CAPIRE

20/03/2026

Algoritmo di Ukkonen 1995

"Online construction of suffix trees"

- con i s.trees sappiamo fare tutto
- come si costruisce un s.tree? tempo? $\Theta(n)$?
- domande: conviene partire con i suffissi lunghi? corti? in che ordine?



Altri elementi della strategia generale

- 1) S.Trie troppo grandi (nel caso pessimo) $\Rightarrow$ S.Tree
- 2) S.Trie e S.Tree saranno automi a stati finiti + f $\leftarrow$ suffix function (funzione ausiliaria)

NOTAZIONE & FORMALIZZAZIONE

$T = t_1 \dots t_n$ $T_i = i\text{-esimo suffisso}$ $T^i = i\text{-esimo prefisso}$
 $\rightarrow$ serve ad evitare di trattare casi limite separatamente (pensato come "suffisso di E ")

$$S_Trie(T) = (Q \cup \{\perp\}, \text{root}, F, g, f)$$

FIN QUI È RIPASSO (19:52 DC)

ci guida nell'aggiornamento da $S_Trie(T^{i-1}) \rightsquigarrow S_Trie(T^i)$

Costruzione ottimale di $Suffix_Trie(T^i)$

Oss.: non sto uscendo automi:
minimi Pensaci a casa

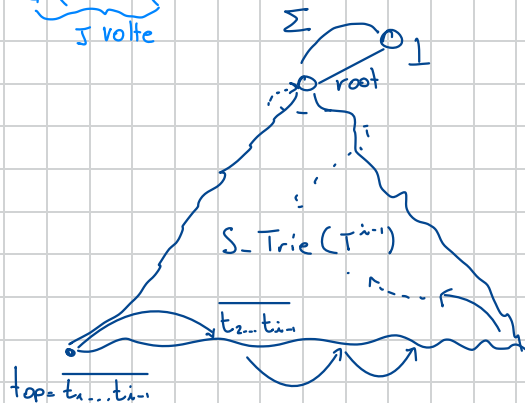
Gli elementi di $\mathcal{S}(T^{i-1})$ si visitano in una lista che parte da:

$\overline{t_1 \dots t_{i-1}} \in Q$ stato corrispondente al suffisso
max di T^{i-1} (e' chiamato top)

Gli stati corrispondenti agli altri f di $\mathcal{S}(T^{i-1})$ saranno del tipo:

Boundary Path $\rightarrow r = f^j(\overline{t_1 \dots t_{i-1}})$ con $j \in \{0, 1, \dots, i-1\}$

$f(f \dots f$
j volte



Non mi muovo
sempre nelle foglie

Il b.p. (boundary path) procederà lungo le foglie fino al primo suffisso che occorre anche come prefisso di un altro suffisso $\rightsquigarrow$ importantissimo da capire

consente di costruire $Suffix_Trie(T^i)$ da $S_Trie(T^{i-1})$

Algorithm: (1)

$r \leftarrow top$ # r visiterà tutto il boundary path

while $g(r, t_i)$ is undefined do

create r' and set $g(r, t_i) = r'$

if $r \neq top$ then create $f(old\ r') = r'$

$old\ r' \leftarrow r'$

$r \leftarrow f(r)$

end while

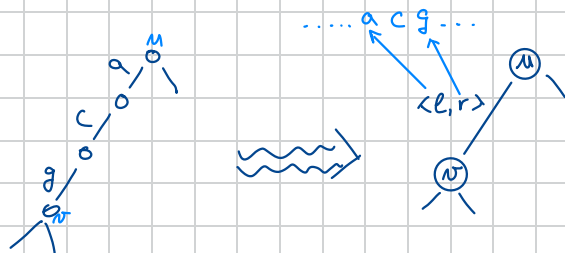
create new suffix link $f(old\ r') = g(r, t_i)$

$top \leftarrow g(top, t_i)$

all'uscita del while
loop ho finito
(gli stati da aggiungere ci
sono già)

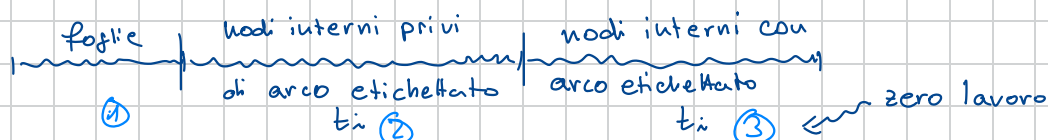
Theo: Suffix_Trie (T) può essere costruito in tempo $O(|T|^2)$ ottimale

Ingredienti per adattare la costruzione al caso dei suffix tree → Suffix Trie nei quali ho eliminato i cammini unitari



- 1) adattamento della definizione di automa a : gli stati saranno solo i nodi branching
- 2) devo adattare l'algoritmo per i Trie

Il punto 2) è delicato: devo dividere il b.p. in tre parti



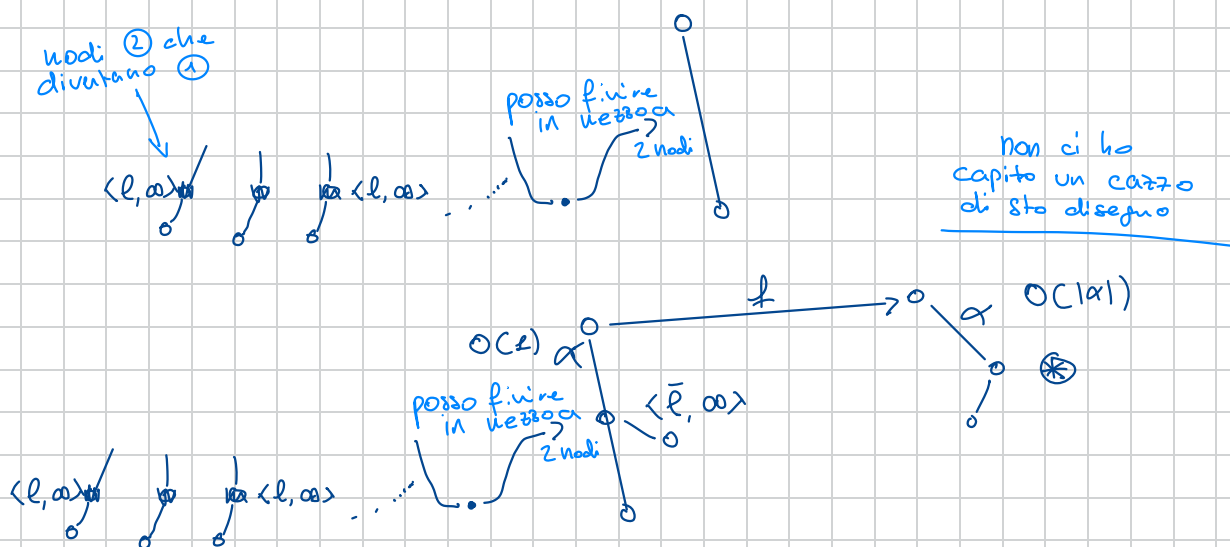
Osservazioni:

- i) "once a leaf always a leaf" le etichette per le foglie saranno di tipo $\langle l, \infty \rangle$; zero lavoro su ①
- ii) Su ② dovrò aggiungere nuovi nodi, che saranno foglie $\Rightarrow$ saranno ① al passo successivo

∞ $\rightarrow 0 \leq \infty$
 ∞ simbolo per fine stringa



$\Rightarrow$ mi rimane da provare che posso operare in ② in tempo lineare nella sua lunghezza



Per dimostrare che su ② eseguo una quantità lineare di passi, osservo che

- a) posso determinare l'arco da dividere e aggiornare in tempo costante
- b) posso aggiungere un nodo e una foglia in tempo costante
- c) posso muovermi usando il suffix link in tempo costante
- d) devo ridiscendere ④ in tempo proporzionale ad α .

Per d) useremo l'analisi ammortizzata.

$Q' \leadsto$ solo branching
 $\uparrow$
 $S_Tree = (Q' \cup \{ \}, root, g', f') \quad \leftarrow \text{mancono gli stati finali ma attualmente non ci servono.}$

- Q' solo nodi branching
- g' transizione da nodo branching a nodo branching
- f' suffix link da nodi branching a nodi branching

Conviene rappresentare i nodi in 2 categorie:

- espliciti (branching)
- impliciti (non branching)

$\hookrightarrow$ gli stati diventano coppie $\langle \overset{\substack{\uparrow \text{stringa}}}{S}, \overset{\substack{\uparrow \text{stato branching}}}{K, p} \rangle \quad \leadsto \text{dove trovo gli indici di } \alpha$

ORA TECNICAMENTE ABBIAMO TUTTI GLI ELEMENTI PER LEGGERE IL DOCUMENTO IN 2 ORE E NON IN DUE GIORNI